TinyML: learning at the sensor edge



Small devices.

Useful decisions.

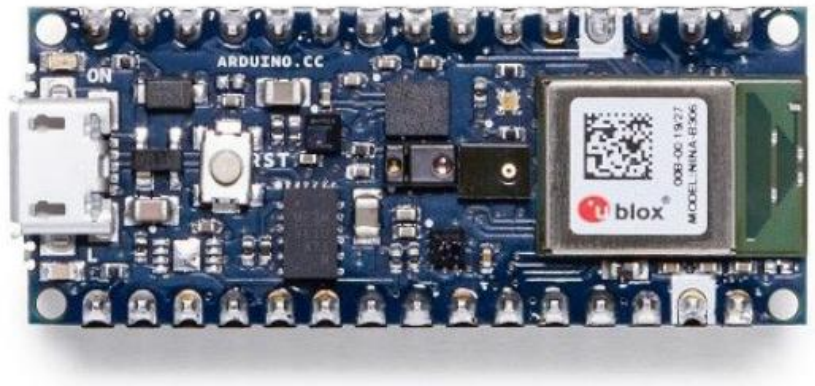
Machine learning meets sensors, embedded systems, and responsible design.

A useful model must fit the device and serve a real application.

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

What makes machine learning “tiny”?



Arduino board shown in the course source

On-device sensor analytics

On the device

Analyze sensor data locally.

Very low power

Typically mW and below in the source definition.

Always-on potential

Enable battery-operated sensing.

TinyML combines on-device sensor analytics with severe resource constraints.

Three disciplines meet in one application

Machine learning

What patterns can be learned from data?

Embedded systems

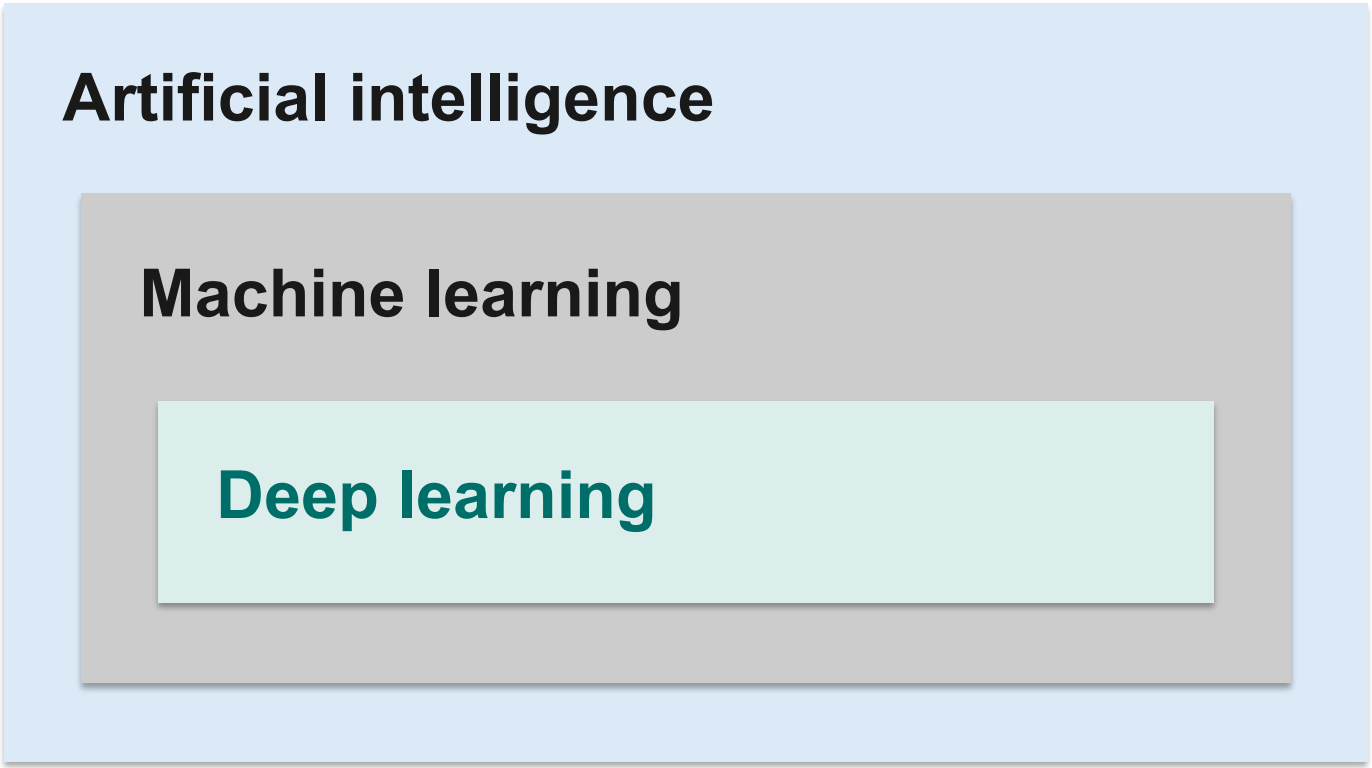
What can run within hardware and software limits?

Applications

What useful task does the device enable?

Learn enough of each discipline to reason about the complete system.

AI contains machine learning and deep learning



ML: algorithms learn patterns by analyzing data.

Deep learning: ML using neural networks.

TinyML is a deployment setting for learned behavior.

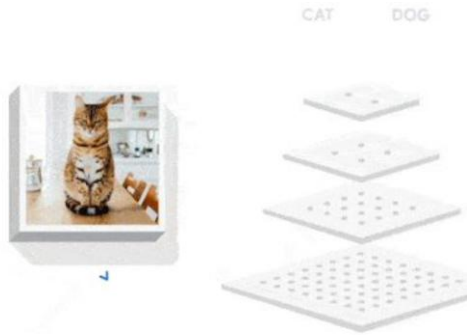
Choose the output your application needs

Task	Output
Image classification	A category for an image
Object detection	Object categories and locations
Segmentation	A category for each pixel
Translation / recommendation	Translated text / suggested items

Different tasks produce different kinds of predictions.

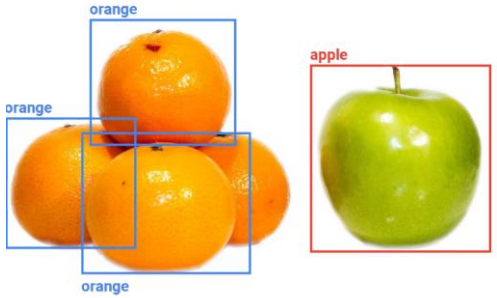
Vision tasks answer different questions

Classification



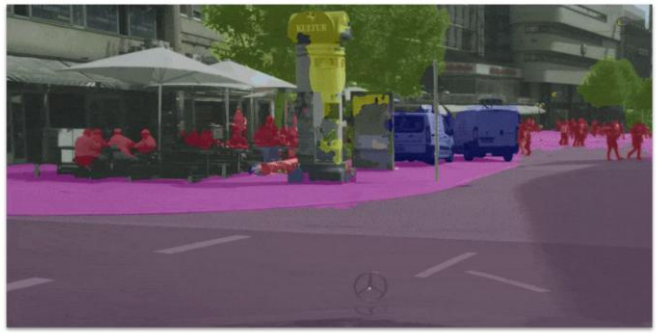
Which category?

Detection



What and where?

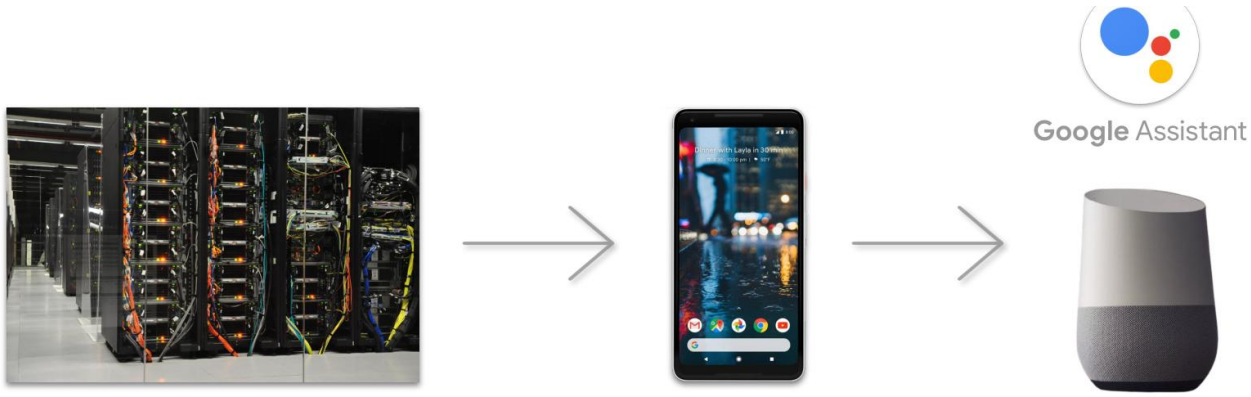
Segmentation



Class of each pixel

Select the output detail that the application actually needs.

Why move ML from cloud to endpoints?



Source illustration: datacenter → phone → endpoint

Power

Avoid unnecessary data movement.

Bandwidth

Process observations near their source.

Latency

Make decisions without a network round trip.

Local processing can reduce communication and response delay.

ElephantEdge: four reasons to sense locally

ElephantEdge

Building The World's Most Advanced Wildlife Tracker.



ElephantEdge illustration from the source

Risk

Movement into high-risk areas

Conflict

Approaching farming areas

Activity

Drinking, eating, or sleeping

Communication

Elephant vocalizations

Start with the monitoring goal, then select sensors and decisions.

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

The basic system: input → process → output

Input

Microphone captures audio

Processing

Interpret input and execute a command

Output

Speaker plays a response

Source example: “Okay, Google!” → “Hi, how can I help?”

A model is one part of the input–processing–output system.

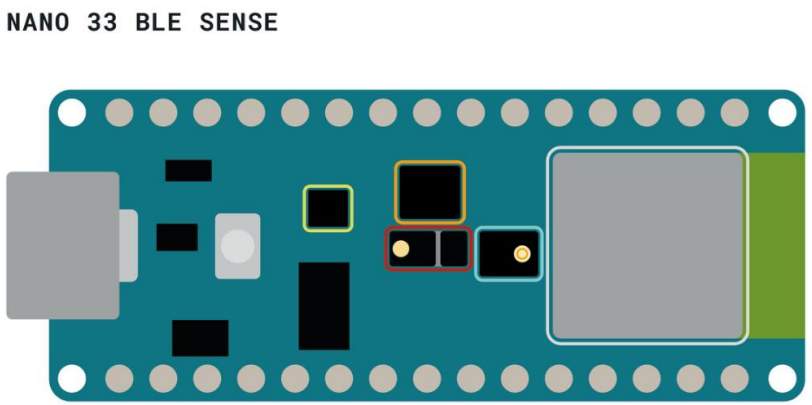
Sensors connect models to the physical world

Family	Examples from the source
Motion / rotation	Accelerometer, gyroscope, magnetometer, encoder
Acoustic / imaging	Microphone, ultrasonic, visible or thermal imaging
Environment / force	Temperature, humidity, pressure, strain
Touch / biometric	Capacitive touch, fingerprint, heart-rate sensing

The available sensor signals determine what the system can observe.

The course board brings several sensors together





Source board: Nano 33 BLE Sense

Sound and motion

Microphone and motion sensing

Environment

Temperature, humidity, and pressure

Light and proximity

Color, brightness, proximity, and gesture

The board combines sensing, computation, and interfaces in one small platform.

Small size, low power, and cost enable TinyML

Source example	Reported characteristic
Kinetis KL03	3.2 mm²; 48 MHz; 32 KB flash
NVIDIA Tesla K80	300 W
Apple A12	3.64 W
Syntiant NDP100	140 µW; 128 KB SRAM; audio recognition

Treat these figures as source-era examples, not current buying advice.

How small can a microcontroller be?



Package dimensions

1.6 × 2.0 mm = 3.2 mm²

Source example

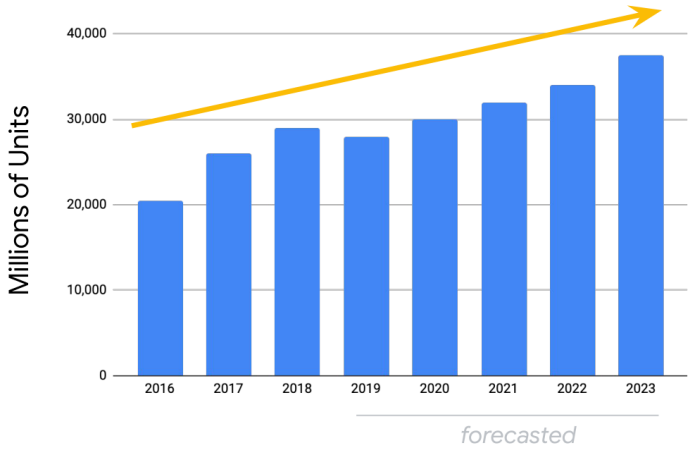
Kinetis KL03 microcontroller

Reported resources

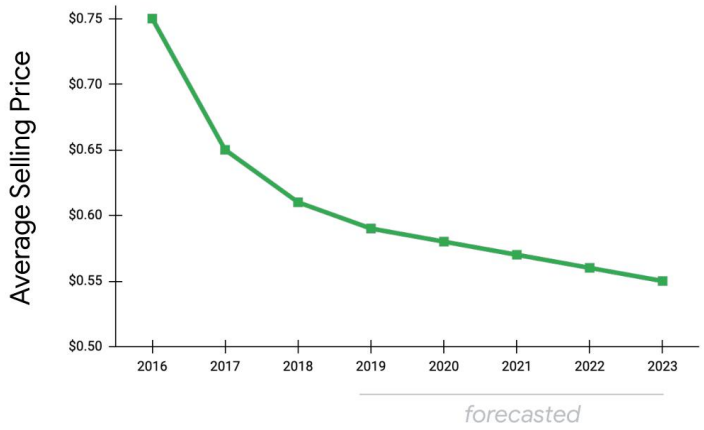
48 MHz; 32 KB flash

A small package still has strict limits on computation and storage.

The source links MCU scale to demand and cost



MCU demand — original source: IC Insights



Average selling price — original source: IC Insights

Historical source charts explain motivation; they are not current market forecasts.

Compute, memory, and storage do different jobs

Compute

Execute instructions.

How complicated is the task?

Memory

Hold working data.

How much space does execution need?

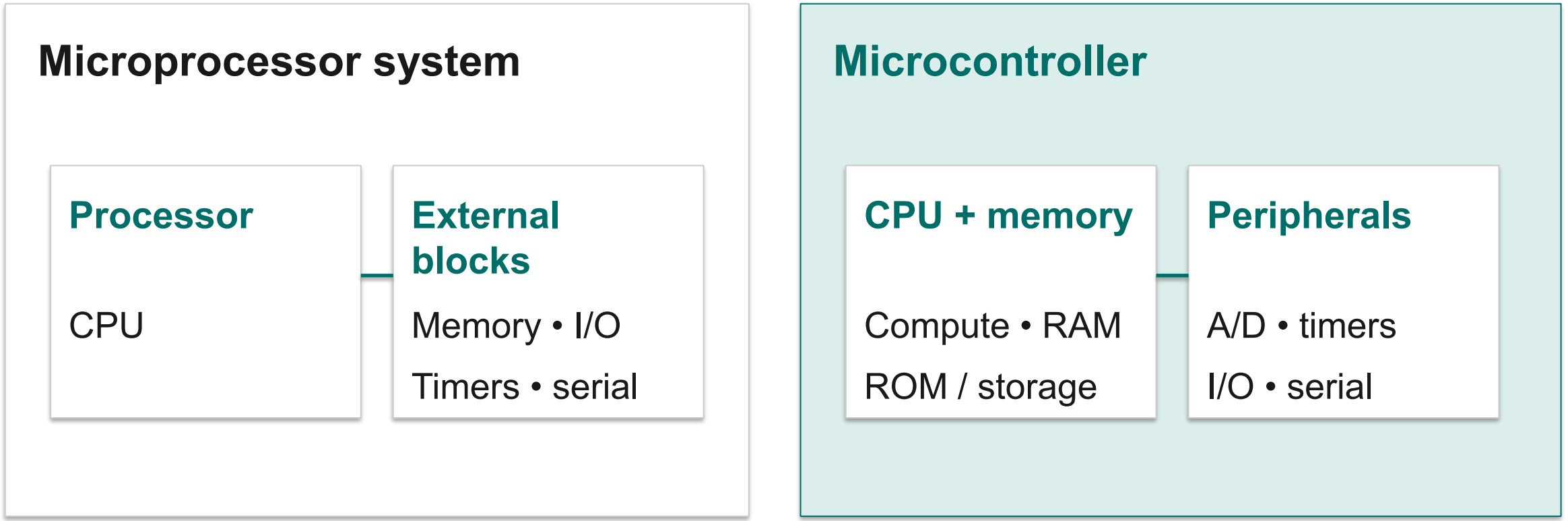
Storage

Retain code and model data.

What must fit on the device?

A task must fit all three resource budgets.

A microcontroller integrates the system blocks



Integration makes compact, specialized embedded systems possible.

The source shows orders-of-magnitude constraints

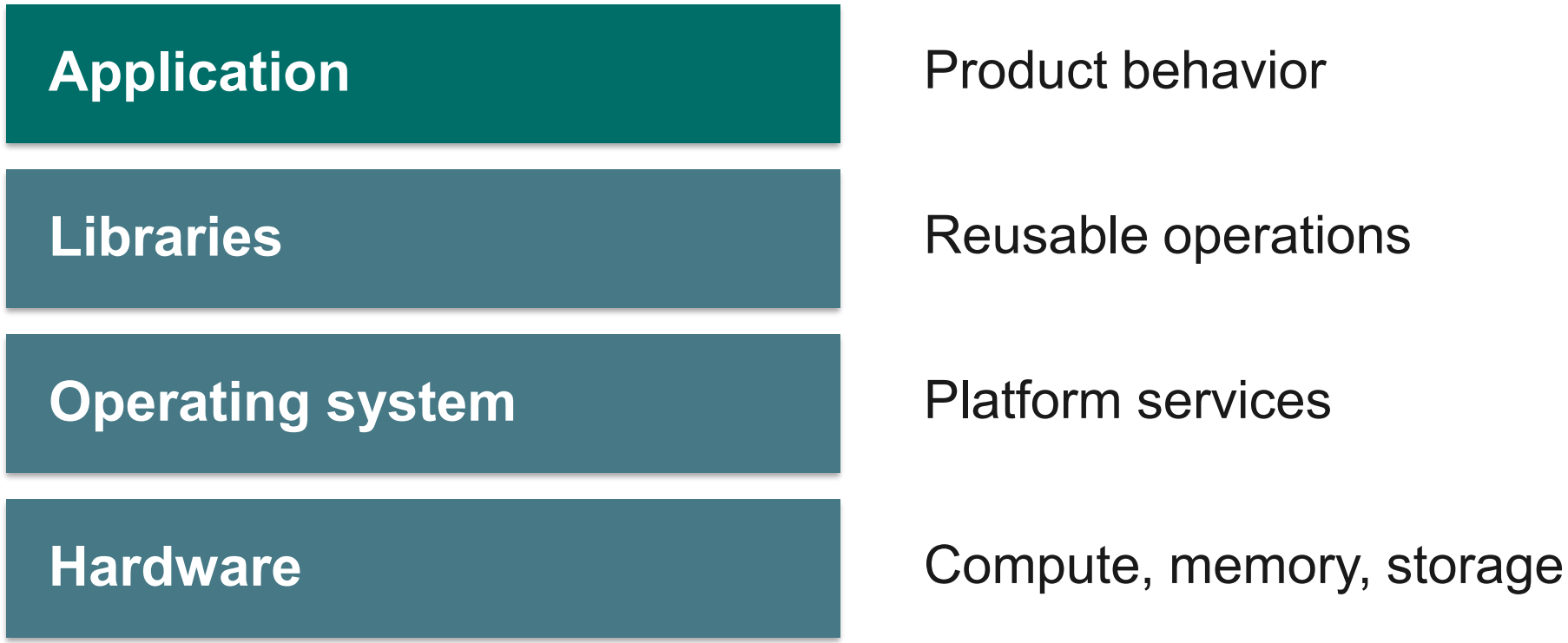
Resource	Microcontroller	Microprocessor system
Clock frequency	1–400 MHz	1–4 GHz
Memory	2–512 KB	512 MB–64 GB
Storage	32 KB–2 MB	64 GB–4 TB
Power	150 µW–23.5 mW	30–100 W

Ask: how much work, how much memory, and how long must it operate?

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

Software sits between the application and hardware



The complete software stack must match the target platform.

Portability trades off against target efficiency

Portable implementation

Reuse code across platforms.

Reduce platform-specific engineering.

Hardware-specific implementation

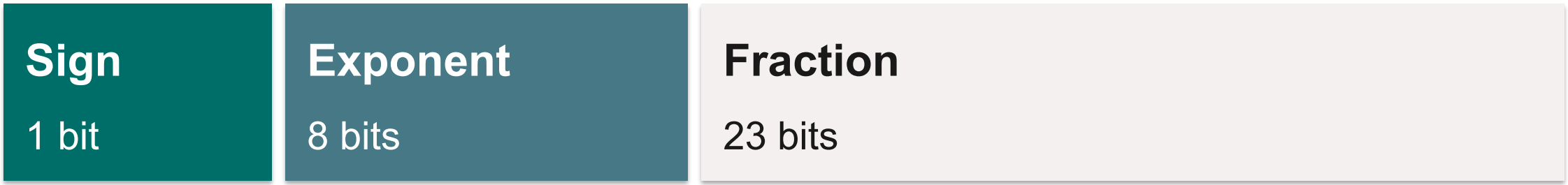
Exploit the target's capabilities.

Spend more effort adapting and validating code.

Choose the implementation that meets the application's resource limits.

Numerical representation affects implementation

π ≈ 3.14159…

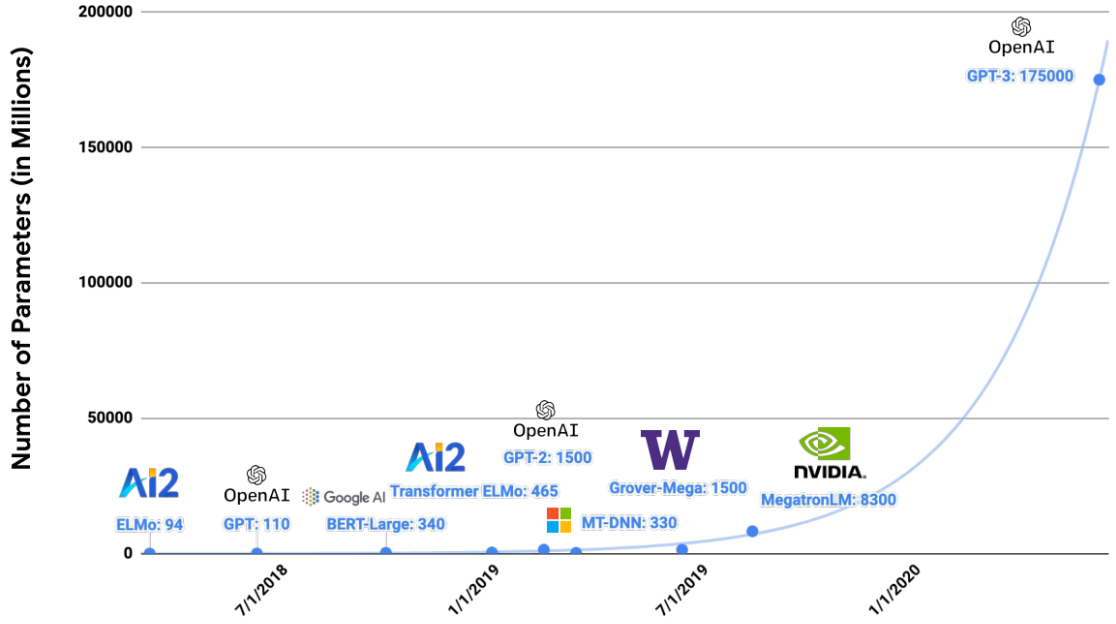


FP32: 32 bits per value

Arithmetic may use software routines or a hardware floating-point unit.

The same numerical operation can have different implementation costs.

Large-model progress creates a tiny-device mismatch



Historical model-parameter growth chart from the source

Model demand

Model growth increases resource needs.

Tiny-device limits

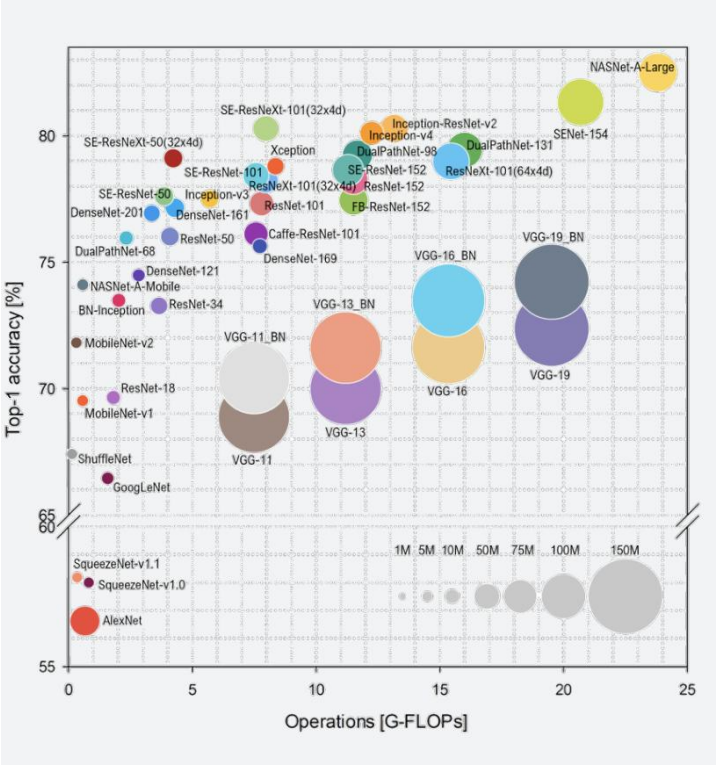
Limited memory, storage, and power

Design response

Use compact architectures and compression.

A more capable model is useful only if the deployment can support it.

Accuracy and computation form a design trade-off



Vertical axis

Top-1 accuracy

Horizontal axis

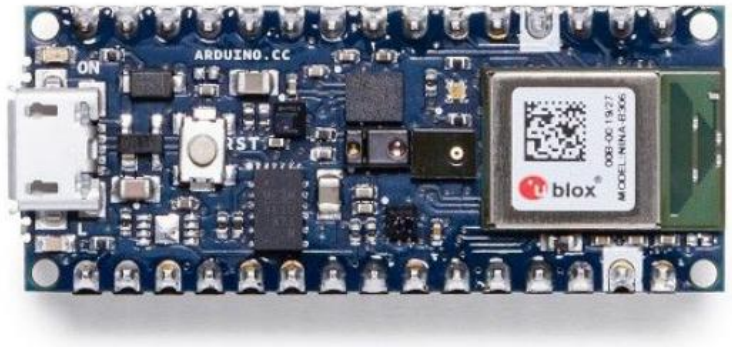
Operations per prediction

Bubble size

Number of model parameters

Compare accuracy, operations, and parameters rather than one number alone.

Model size and working RAM are not interchangeable



Course board: 256 KB RAM

Model example

MobileNetv1: 16.9 MB reported size

Storage ≠ working RAM

They are different resource budgets.

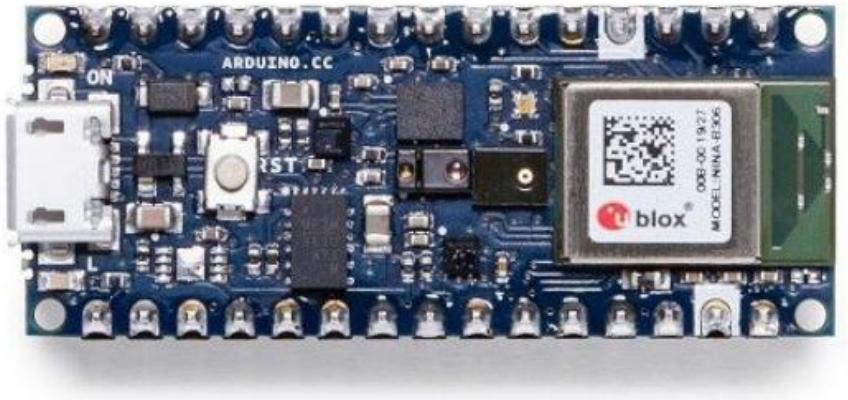
Check execution needs

Account for working data and software state.

Check both model storage and the memory needed during execution.

Training learns a model; inference uses it





Training: learn weights

Data → learned model

Inference: use weights

New input → prediction

A large training environment can produce a small inference model.

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

Three ways to make a model smaller

Pruning

Remove connections or neurons.

Quantization

Represent values with fewer bits.

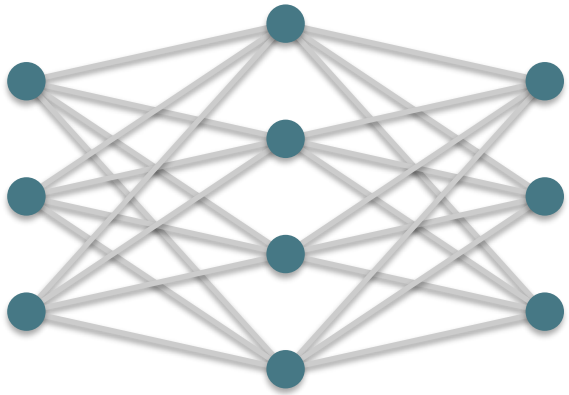
Distillation

Transfer behavior from teacher to student.

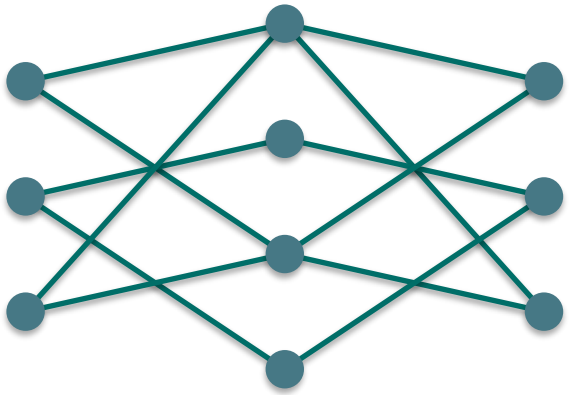
Compression changes the model; assess the resulting behavior.

Pruning removes connections or whole neurons

Original network

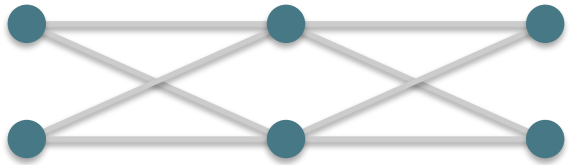


Fewer connections



Connection pruning

Fewer neurons



Neuron pruning

Pruning changes the network's connectivity or structure.

Quantization maps values to a lower-precision grid

Float32

32 bits per value

→

Map values

Approximate on an integer grid

→

Int8

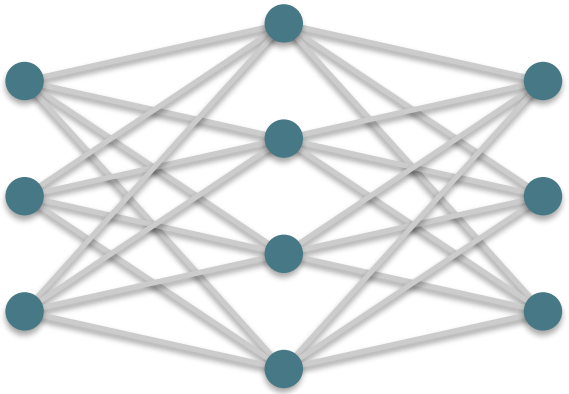
8 bits; codes −128 to 127

Fewer bits reduce storage per value and change numerical precision.

32 → 8 bits gives a 4× smaller raw payload for the same number of values.

Distillation transfers behavior to a smaller student

Teacher

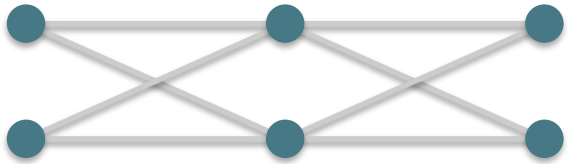


Larger network

Learning signal



Student



Smaller network

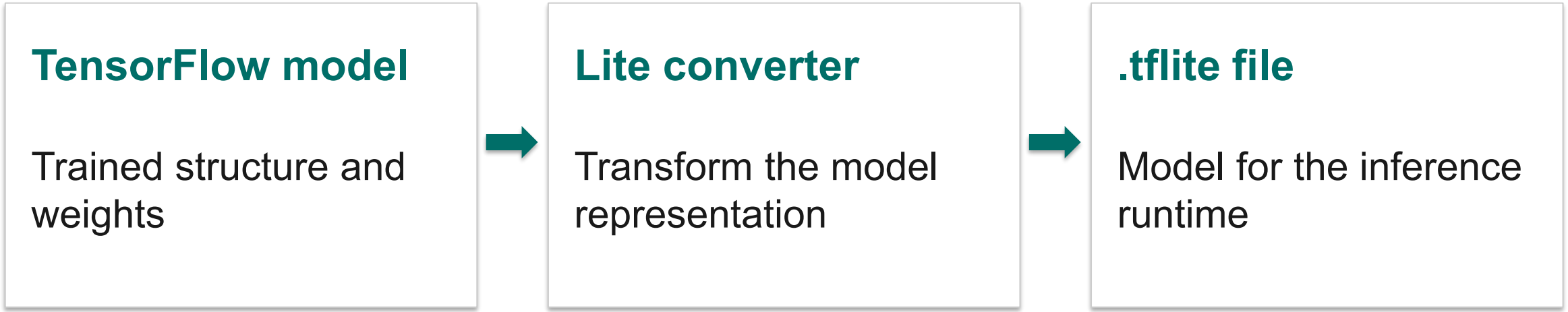
The student is the compact model intended for deployment.

Inference runtimes prioritize a smaller footprint

Concern	Training environment	Deployed inference
Model structure / weights	Can change during development	Fixed for the deployed model
Binary footprint	More flexibility available	High priority
Compute environment	May use distributed compute	Local execution
Primary user in source	ML researcher	Application developer

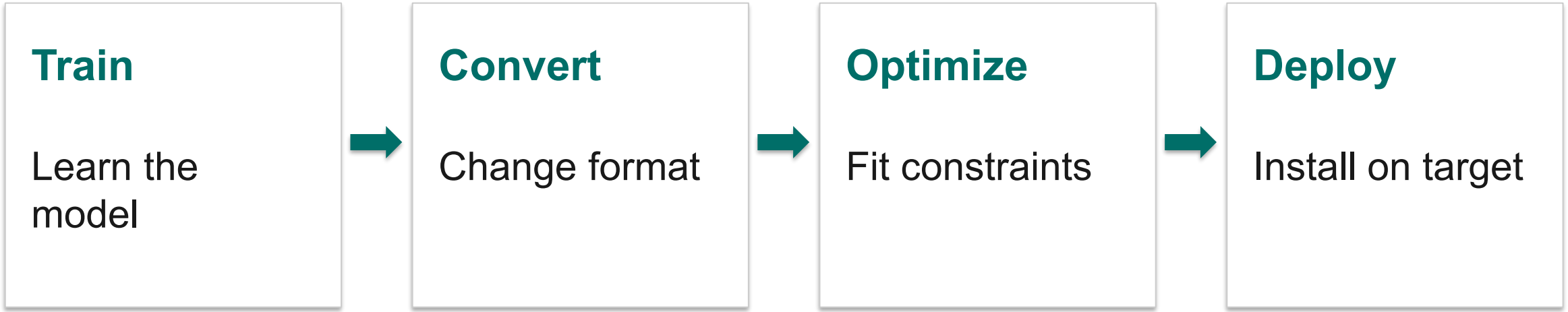
Runtime design is part of making the model fit.

Convert the trained model into a deployment format



A model file, runtime, and hardware must work together.

The workflow ends with inference at the edge



Then make inferences at the edge: Linux systems, Raspberry Pi, or microcontrollers.

Deployment joins machine learning models, runtimes, and hardware.

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

Responsible AI asks what happens beyond prediction

Practices

How does this system change the way people work?

Decisions

Who acts on its predictions, and how?

Behavior

Who is observed, influenced, or constrained?

Consider benefits and consequences throughout the product lifecycle.

Industry: useful alarms must support action



Benefit

Reduce downtime and improve efficiency.

Accuracy

Balance missed events and false alarms.

Operator response

Make alarms understandable and actionable.

Evaluate missed detections, false alarms, and operator response together.

Environment: protect the observations you collect



Benefit

Gain insights and waste less sensor data.

Reliability

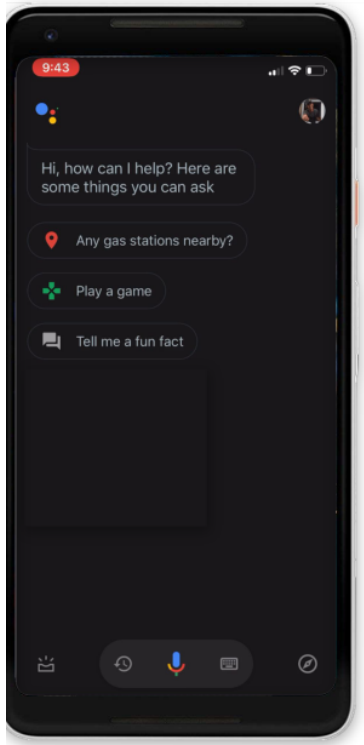
Check observations and decisions.

Security

Control access and prevent misuse.

Useful environmental data can also be sensitive data.

Human interfaces: accessibility requires reliability



Benefit

Support hands-free interaction.

Reliability

Evaluate performance across users.

Privacy

Protect the data the device collects.

Check performance across users and protect their data.

Keep human values in every lifecycle stage

Design

What are we building?

For whom?

What if it fails?

Development

What training data?

What bias?

How will fairness be assessed?

Deployment

Monitor model drift.

Address security breaches.

Protect privacy.

Responsible AI is proactive work, not a final checklist.

Lecture Overview

- 1 What is TinyML?
  Definition, core ideas, ML tasks, and why ML moves to the edge.
- 2 Hardware
  Sensors, microcontrollers, and what limits them.
- 3 Software and models
  Embedded software, model constraints, and the model lifecycle.
- 4 Making models fit
  Compression, runtimes, conversion, and deployment.
- 5 Responsible AI
  Industry, environment, people, and human-centered design.
- 6 Recap
  The three questions every TinyML design must answer.

A TinyML design must answer three questions

What should it do?

Choose sensor inputs and a useful output.

Can it fit?

Check hardware, software, model size, and runtime.

Is it responsible?

Assess failure consequences, fairness, privacy, and security.

Connect the application, feasible implementation, and consequences.